Extension:

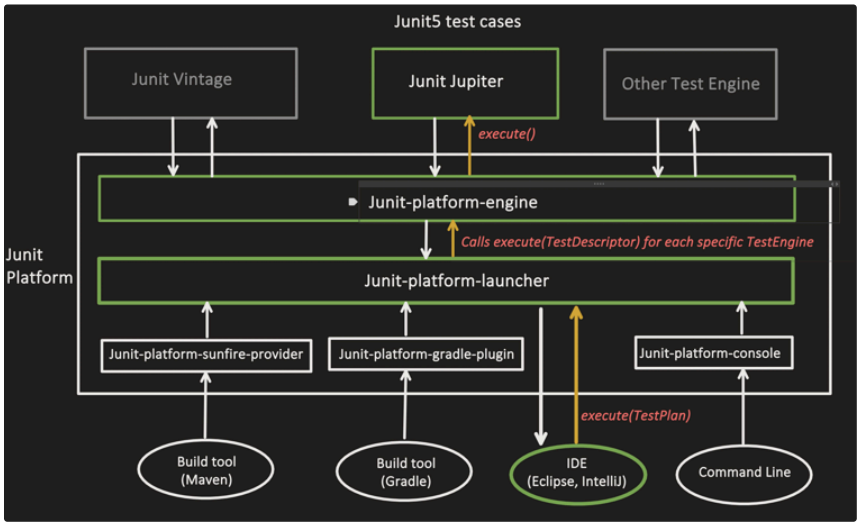
Extension:

In simple term, It allows us to add custom behavior to tests.

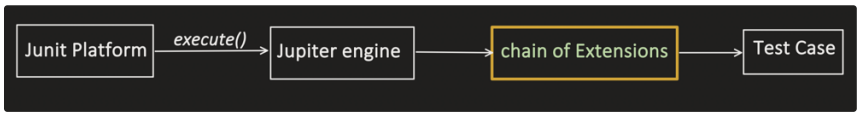
like:

- Enable or disable test conditionally.
- Add custom code before or after tests.
- Intercept Method calls for logging etc.

Remember this diagram from the "Architecture" topic:



Where does Extensions fits in:



Lets first go through each extension first, then at end, will show the dummy flow to visualize the ORDERING of JUnit5 extensions. That will clarify lot of things.

Some of the Extensions Types are:

- Lifecycle Extensions
- Parameter Resolver
- Execution Condition
- Test Execution Callbacks (Before and After)
- Invocation Interceptor
- Exception Handler

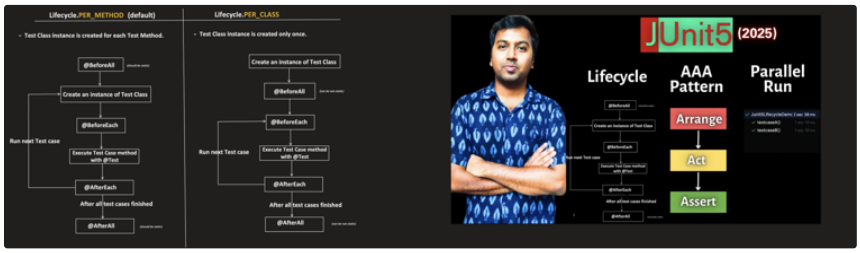
Concept && Coding

- Instance Post Processor

1. **Lifecycle Extensions:**

- Similar to lifecycle annotations "@BeforeAll, @BeforeEach, @AfterAll, @AfterEach".
- Lifecycle Extensions are used to execute custom logic before/after test classes or methods.

We have already covered this Lifecycle topic, kindly check it out, if there is any doubt with this:



One question might comes to your mind: Why we need lifecycle extensions, when we already have annotations like @BeforeAll, @BeforeEach, @AfterAll and @AfterEach?

These lifecycle methods withs annotations @BeforeAll, @BeforeEach, @AfterAll and @AfterEach are tied to a single class.

While

Lifecycle extensions are reusable across many test classes.

Our custom class implemented an BeforeAllCallBack extension interface:

```
public class BeforeAllLifecycleExtension implements BeforeAllCallback {

    @Override
    public void beforeAll(ExtensionContext context) {
        System.out.println("Extension: Before all tests");
    }
}

```

Our custom class implemented an AfterAllCallBack extension interface:

```
public class AfterAllLifecycleExtension implements AfterAllCallback {

    @Override
    public void afterAll(ExtensionContext context) throws Exception {
        System.out.println("Extension: After all tests");
    }
}

```

Execution Context object has all the info about:

- ***Test class*** → *via getTestClass()*
- ***Test method*** → *via getTestMethod()*
- ***Annotations*** → *accessible through getElement()*

Concept && Coding

- ***Test instance*** → *via getTestInstance()*

Registering multiple Lifecycle extension (comma separated) within a test class:

```java
@ExtendWith({BeforeAllLifecycleExtension.class, AfterAllLifecycleExtension.class})
class MyServiceTest {

    MyServiceTest() {
        System.out.println("instance created");
    }

    @BeforeAll
    public static void beforeAllMethod() {
        System.out.println("inside beforeAll");
    }

    @BeforeEach
    public void beforeEachMethod() {
        System.out.println("inside beforeEach");
    }

    @Test
    void testMethod1() {
        System.out.println("inside test method");
    }

    @AfterEach
    public void afterEachMethod() {
        System.out.println("inside afterEach method");
    }

    @AfterAll
    public static void afterAllMethod() {
        System.out.println("inside afterAll method");
    }
}

```

Or we can also write one class , which implements all the lifecycle callback extension interface:

```java
public class LifecycleExtension implements BeforeAllCallback, AfterAllCallback,
        BeforeEachCallback, AfterEachCallback {

    @Override
    public void beforeAll(ExtensionContext context) {
        System.out.println("Extension: Before all tests");
    }

    @Override
    public void afterAll(ExtensionContext context) throws Exception {
        System.out.println("Extension: After all tests");
    }

    @Override
    public void afterEach(ExtensionContext context) throws Exception {
        System.out.println("Extension: After each tests");
    }

    @Override
    public void beforeEach(ExtensionContext context) throws Exception {
        System.out.println("Extension: Before each tests");
    }
}
```

Concept && Coding

```java
@ExtendWith(LifecycleExtension.class)
class MyServiceTest {

    MyServiceTest() {
        System.out.println("instance created");
    }

    @BeforeAll
    public static void beforeAllMethod() {
        System.out.println("inside beforeAll");
    }

    @BeforeEach
    public void beforeEachMethod() {
        System.out.println("inside beforeEach");
    }

    @Test
    void testMethod1() {
        System.out.println("inside test method");
    }

    @AfterEach
    public void afterEachMethod() {
        System.out.println("inside afterEach method");
    }

    @AfterAll
    public static void afterAllMethod() {
        System.out.println("inside afterAll method");
    }
}
```

Output:

```
Extension: Before all tests
inside beforeAll
instance created
Extension: Before each tests
inside beforeEach
inside test method
inside afterEach method
Extension: After each tests
inside afterAll method
Extension: After all tests

Process finished with exit code 0
```

2. **Parameter Resolver Extension:**

- With this, we can dynamically supplies values to method or constructor parameters within a test class.

```java
public class Student {

    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
```

Concept && Coding

```
    }
}

```

```
@ExtendWith(StudentParameterResolver.class)
class MyServiceTest {

    @Test
    void testMethod1(Student student) {
        System.out.println("Student name is: " + student.name);
    }

}

```

@ExtendWith(StudentParameterResolver.class): ***At class level, this extension will be applied to all the method and constructor.***

We can use it at method level too, then this extension will be applied to that particular method only.

```
class MyServiceTest {

    @Test
    @ExtendWith(StudentParameterResolver.class)
    void testMethod1(Student student) {
        System.out.println("Student name is: " + student.name);
    }
}
```

```
public class StudentParameterResolver implements ParameterResolver {

    @Override
    public boolean supportsParameter(ParameterContext parameterContext,
                                     ExtensionContext extensionContext)
            throws ParameterResolutionException {

        return parameterContext.getParameter().getType() == Student.class;
    }

    @Override
    public Object resolveParameter(ParameterContext parameterContext,
                                   ExtensionContext extensionContext)
            throws ParameterResolutionException {

        return new Student("myName", 27);
    }
}

```

ParameterContext object has all the info about the particular parameter:

- *Its position in parameter list*
- *Its type*
- *Which method or constructor declared this variable*

<u>Output:</u>

Concept && Coding

```
MyServiceTest (com.cc 10 ms
    testMethod1(Studen 10 ms
Tests passed: 1 of 1 test – 10 ms
/Library/Java/JavaVirtualMachines/jdk-17.jd
instance created
Student name is: myName

Process finished with exit code 0
```

3. **Execution Condition Extension:**

- This extension, helps to decide whether a test should be enabled or disabled at runtime based on some custom logic.

Example:

Junit-platform.properties:

```
junit-platform.properties
#below configurations, junit platform launcher will apply to junit.jupiter (i.e. Jupiter Test engine only)

xyz.feature.enabled=false
```

```java
public class ConditionalExecutionExtension implements ExecutionCondition {

    @Override
    public ConditionEvaluationResult evaluateExecutionCondition(ExtensionContext context) {

        boolean featureEnabled = Boolean.parseBoolean(

context.getConfigurationParameter("xyz.feature.enabled").orElse("false")
        );

        return featureEnabled
                ? ConditionEvaluationResult.enabled("feature enabled, test can run")
                : ConditionEvaluationResult.disabled("feature disabled, test can not run");
    }
}
```

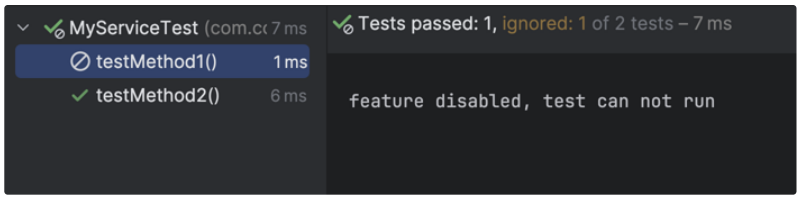
@ExtendWith(ConditionalExecutionExtension.class): *Used on top of method, so this extension only applied to this particular method. If used on top of class, it will be apply to all the method of this class.*

```java
class MyServiceTest {

    @Test
    @ExtendWith(ConditionalExecutionExtension.class)
    void testMethod1() {
        System.out.println("inside testMethod1");
    }

    @Test
    void testMethod2() {
        System.out.println("inside testMethod2");
```

Concept && Coding

```
    }
}

```

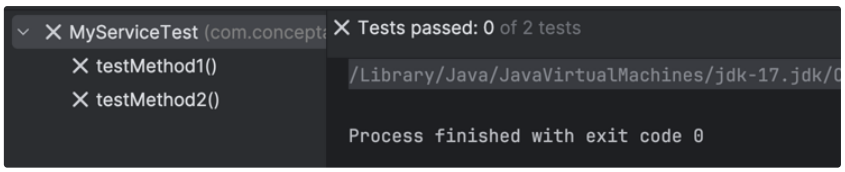
Output:



Using the extension on top of the class:

```
@ExtendWith(ConditionalExecutionExtension.class)
class MyServiceTest {

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1");
    }

    @Test
    void testMethod2() {
        System.out.println("inside testMethod2");
    }
}

```

<u>Output (feature is currently disabled):</u>



4. **<u>Test Execution Callback Extension:</u>**

- With this, we can add code logic before or after a test method executes.

- Same thing we can do with @BeforeEach/ @AfterEach method, but they have setup logic, its not good idea to mess it with custom logic like recording the execution time etc..

  BeforeTestExecutionCallback *method runs, just before the invocation of the test method (after BeforeAllCallback, @BeforeAll, BeforeEachCallback, @BeforeEach*

```
@Test
void testMethod1() {

    System.out.println("inside testMethod1");
}

```

Concept && Coding

AfterTestExecutionCallback *method runs, just after the invocation of the test method (before @AfterEach, AfterEachCallback, @AfterAll, AfterAllCallback*

```java
public class TestExecutionExtension implements
BeforeTestExecutionCallback,
        AfterTestExecutionCallback {

    @Override
    public void beforeTestExecution(ExtensionContext context) {
        System.out.println("Starting test at: " + Instant.now());
    }

    @Override
    public void afterTestExecution(ExtensionContext context) {
        System.out.println("Finished test at: " + Instant.now());
    }
}

```

@ExtendWith(TestExecutionExtension.class): *We can also use on top of particular test method, if we don't want to get it applied for all test methods.*

```java
@ExtendWith(TestExecutionExtension.class)
class MyServiceTest {

    @BeforeEach
    void beforeEachMethod() {
        System.out.println("inside beforeEach method");
    }

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1");
    }

    @AfterEach
    void afterEachMethod() {
        System.out.println("inside afterEach method");
    }
}

```

Output:

```
MyServiceTest (com.c 11 ms        Tests passed: 1 of 1 test – 11 ms
  testMethod1()      11 ms        /Library/Java/JavaVirtualMachines/jdk-17.jdk/Conten
                                  inside beforeEach method
                                  Starting test at: 2025-09-09T10:52:09.188319Z
                                  inside testMethod1
                                  Finished test at: 2025-09-09T10:52:09.190149Z
                                  inside afterEach method

                                  Process finished with exit code 0
```

5. **<u>Invocation Interceptor Extension:</u>**

- It helps to wrap the execution of test method.

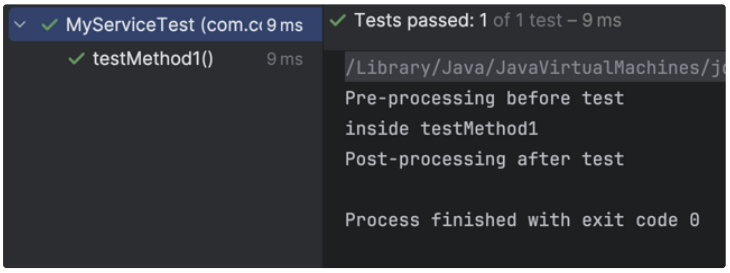
Concept && Coding

- So we can write:
  - Pre processing logic
  - Post processing logic

```java
public class CustomInvocationExtension implements
InvocationInterceptor {

    @Override
    public void interceptTestMethod(Invocation<Void> invocation,

ReflectiveInvocationContext<Method> context,
                                    ExtensionContext extensionContext)
throws Throwable {

        System.out.println("Pre-processing before test");

        invocation.proceed();   // actual test runs here

        System.out.println("Post-processing after test");
    }
}

```

Used at class level:

```java
@ExtendWith(CustomInvocationExtension.class)
class MyServiceTest {

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1");
    }
}

```



Used at method level:

```java
class MyServiceTest {

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1");
    }

    @Test
    @ExtendWith(CustomInvocationExtension.class)
    void testMethod2() {
        System.out.println("inside testMethod2");
    }
}

```

Concept && Coding

```
MyServiceTest (com.conceptandcoding.orderservice) 10 ms        0 ms
  testMethod1()  10 ms
  testMethod2()

/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home
inside testMethod1
Pre-processing before test
inside testMethod2
Post-processing after test

Process finished with exit code 0
```

6. **Exception Handler Extension:**

- It help us to intercept the exceptions thrown by test method and handle it in our own way.

```java
public class CustomExceptionHandler implements TestExecutionExceptionHandler {

    @Override
    public void handleTestExecutionException(ExtensionContext context, Throwable throwable)
            throws Throwable {

        System.out.println("Exception occurred in test: " + context.getDisplayName());
        System.out.println("Exception message: " + throwable.getMessage());

        throw throwable;
    }
}

```

```java
@ExtendWith(CustomExceptionHandler.class)
class MyServiceTest {

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1");
        int x = 1 / 0;
    }
}

```

```
MyServiceTest (com.c 10 ms        Tests failed: 1 of 1 test – 10 ms
  testMethod1()  10 ms

/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/bin/java ...
inside testMethod1
Exception occurred in test: testMethod1()
Exception message: / by zero

java.lang.ArithmeticException: / by zero

    at com.conceptandcoding.orderservice.MyServiceTest.testMethod1(MyServiceTest.java:13)
```

7. **Instance Post Processor Extension:**

- It help to manipulate or initialize the test instance after it has been created.
- It runs before any Lifecycle method or extension runs or even the test method runs.

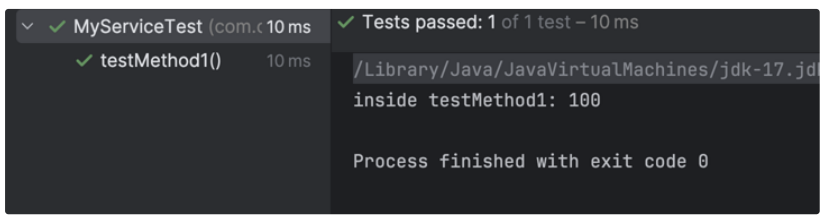
```java
public class CustomPostProcessorExtension implements TestInstancePostProcessor {

```

Concept && Coding

```
    @Override
    public void postProcessTestInstance(Object testObj, ExtensionContext context) {
        if (testObj instanceof MyServiceTest) {
            MyServiceTest myServiceTestObj = (MyServiceTest) testObj;
            myServiceTestObj.value = 100;
        }
    }
}
```

@ExtendWith(CustomPostProcessorExtension.class): *We Can also use it at method level (for PER_METHOD lifecycle, as it creates new instance for each test method)*

```
@ExtendWith(CustomPostProcessorExtension.class)
public class MyServiceTest {

    public int value;

    @Test
    void testMethod1() {
        System.out.println("inside testMethod1: " + value);
    }
}
```



Dummy flow of Jupiter engine to visualize the ORDERING of JUnit5 extensions:

```
public void executeTestClass(Class<?> testClass) {


```

Concept && Coding

```java
    // 1. run BeforeAllCallbacks extension and @BeforeAll
    for (BeforeAllCallback beforeAll : beforeAllCallbacks) {
                        beforeAll.beforeAll();
    }

    //  execute each test method
    for (Method testMethod : testClass.getDeclaredMethods()) {
                if (isTestMethod()) {
                        Object testInstance = createTestInstance();
                         executeTestMethod();
               }
    }

    // 10. AfterAllCallbacks extension and @AfterAll
    for (AfterAllCallback afterAll : afterAllCallbacks) {
                  afterAll.afterAll();
      }
}

```

```java
public void executeTestMethod() {

    try {
        // 2. TestInstancePostProcessor
        for (TestInstancePostProcessor pip : postProcessors) {
                    pip.postProcessTestInstance();
        }

        // 3. BeforeEachCallback  extension and @BeforeEach
        for (BeforeEachCallback beforeEach : beforeEachCallbacks) {
            beforeEach.beforeEach(context);
        }

        // 4. ParameterResolvers extension will run
        int argumentsCount = testMethod.getParameterCount();
        for (int i = 0; i < argumentsCount; i++) {
            Parameter param = testMethod.getParameters()[i];
                        for (ParameterResolver resolver : parameterResolvers) {
                                    if (resolver.supportsParameter(param, context)) {
                                                arguments[i] = resolver.resolveParameter(param, context);
                                                 break;
                                    }
                        }
        }

        // 5. BeforeTestExecutionCallback execution will run
        for (BeforeTestExecutionCallback beforeTest : beforeTestExecutionCallbacks) {
            beforeTest.beforeTestExecution(context);
        }

        // 6. InvocationInterceptor wraps the test method
        invocationInterceptor.interceptTestMethod(() -> {testMethod.invoke(testInstance, arguments);}, testMethod, context);

        //  7. AfterTestExecutionCallback
        for (AfterTestExecutionCallback afterTest : afterTestExecutionCallbacks) {
            afterTest.afterTestExecution(context);
        }

    } catch (Throwable t) {
        //  8. TestExecutionExceptionHandler
        for (TestExecutionExceptionHandler handler : exceptionHandlers) {
            handler.handleTestExecutionException(context, t);
        }
    } finally {
```

Concept && Coding

```
        // 9. @AfterEach and AfterEachCallback extensions
        for (AfterEachCallback afterEach : afterEachCallbacks) {
            afterEach.afterEach(context);
        }
    }
}

```

Concept && Coding